OWASP ASVS is a list of security requirements for web applications and services. It is a valuable resource for anyone designing, developing and maintaining applications or evaluating their security. ASVS stands for *Application Security Verification Standard*. The scope of the ASVS is decided by its name, application, security, verification and standard. ASVS defines an application as a software product being developed, with security controls being implemented. The ASVS does not look at development lifecycles or how applications are built. Instead, it focuses on the security results that must be achieved by the application. According to the ASVS the "application" is referring to the application and any components that work with HTTP traffic such as firewalls. However, they don't include anything outside the systems' control, such as DNS problems. The second requirement of ASVS is security. Each requirement in the ASVS has a verifiable impact on security. Any requirement that is not present results in a less secure application. All focus on functional aspects, code style or policy requirements is out of scope for this section. Verification is the third requirement of ASVS. All parts of the ASVS must be verifiable with a "pass" or "fail" decision. This means that no requirement in the ASVS can be vague or hard to test. ASVS serves as a standard. It is a collection of security requirements that are to be implemented to comply with a standard. The ASVS does not overlap with other OWASP content. For example, ASVS does not talk about "how to implement a particular security requirement" or "how to test a requirement". These questions are answered in other OWASP documentation. When ASVS calls something a requirement it means specifically, "what must be achieved to satisfy the requirement" (OWASP Application Security Verification Standard (ASVS), Page 11, OWASP Foundation, n.d.).

The ASVS has three levels of security verification. Each level has increasing depth and complexity. The aim is for an organization to start at the first level, which focuses on the most

important security needs, and move up to higher levels based on the needs of the organization. The levels in ASVS are L1, L2 and L3. Level 1 contains the minimum requirements. This level is about twenty percent of the requirements of the ASVS. It has an intentionally low barrier to entry. The requirements in this level are the critical and basic defense requirements. The goal of level 1 is to prevent common attacks and get the system ready to advance to level 2. The second level of the ASVS is where most applications should aim to be. Level 2 is fifty percent of the requirements. This means the application must implement seventy percent of the ASVS requirements (All of L1 and L2) to be considered L2. The requirements in this level relate to less common attacks or more complicated security. Finally, level 3. Level 3 is the final thirty percent of the ASVS. This level looks to implement the highest level of security. Things like defense-in-depth mechanisms or other hard-to-implement controls.

Not every application needs to get to level 3. ASVS does not tell you what level your organization should be at. Instead, it is up to the organization to determine what level they believe they should be. A new startup company should focus on level 1 for its security. A well-established bank would have a hard time arguing for anything less than level 3. The ASVS is an incredibly good tool for application security. However, it isn't perfect. According *to A quantitative security evaluation and analysis model for web applications based on OWASP application security verification standard*, ASVS lacks the capability to diagnose at a granular level. "A granular-level diagnosis goes beyond surface-level assessments and delves into a more comprehensive and meticulous analysis of various facets of the system." (Wen & Katt, 2023b). Basically, ASVS does not consider granular reasons for an application being set up in certain ways. This is more impactful for an organization that has specialized systems that need to be

configured in specific ways. The ASVS is a guideline. It serves to get an organization to a secure position.

Static and dynamic analysis are two different ways of testing a system for vulnerabilities. Static analysis focuses on source code or binary code without executing the program. The advantage of static analysis is the ability to detect vulnerabilities early in the development process. Static analysis does not disrupt system operations. However, static analysis cannot detect errors that appear when the program is running and may produce false results. Dynamic testing analyses data when the program is running. This allows dynamic tests to find vulnerabilities in the runtime and test security strength in real environments. However, dynamic tests require a high level of expertise and disrupt system operations. The best option is to use static and dynamic testing together. “In a study on phishing website detection, the authors created a tool that combines static analysis to extract features from HTML code with dynamic analysis by rendering the web page in a virtual browser… this combined approach reduced the number of difficult-to-classify samples by 16%.” (Cong et al., 2024). When combining static and dynamic testing the process can be described as:

1. Source code review: Start with a thorough analysis of the source code. This will identify vulnerabilities and logical flaws that can’t be seen from the outside.
2. Targeted dynamic tests: With the information gained from the static tests. Perform dynamic tests on areas of concern.
3. Document and report: Document the vulnerabilities found, including the way they were discovered and the potential impact.

Integrating static and dynamic tests gives access to the strengths of both approaches. Static analysis helps identify errors in the source code without running the program, while dynamic

analysis can detect issues that only appear while the program is running. This is especially important for modern web applications where the complex nature of web applications makes individual methods less effective.

In the following sections there is an example of a security testing plan. It incorporates manual source code reviews, automated source code reviews, automated dynamic testing and penetration testing. The purpose of the security test plan is to provide an understanding of how and why these security tests are used. The strategy proposed in this plan is a multi-layered testing approach. Layering the different tests to allow for the most coverage of possible vulnerabilities.

The first leg of the security testing plan is manual source code review. Source code reviews aim to systematically assess code to make sure it follows specific standards, such as ASVS. Code reviews often focus on the software architecture, functionality and style but can also deal with security. Manual source code reviews often catch logic or design errors, architecture flaws, security vulnerabilities and maintainability issues. The advantage of manual source code reviews is timing. These reviews can be performed at any time during the development of the application. This means problems can be found much earlier in development. These tests are important to our plan because they allow us to find vulnerabilities early on in development. Vulnerabilities are not being found days before release, they are found days after the code is written. Manual code reviews also find readability issues. Places where the code isn't clear for an external observer. We will be using manual source code reviews throughout the development process. Using them to determine places where other testing needs to be performed.

The second leg of the security plan is automated source code reviews. Automated source code reviews utilize scanners that can identify security vulnerabilities. They operate in a similar way to manual source code reviews. These automated scanners or SAST (Static Application

Security Testing) scanners look at the architecture, functionality, style and security of the application. These SAST scanners have some advantages over manual reviews. They are faster and more efficient. They can scan large amounts of code quickly, making them much faster and more efficient than the manual reviews. They are more consistent than manual reviews. Where manual reviews suffer from inconsistencies from the knowledge of the person doing the review, SAST scanners are designed to be consistent. SAST scanners also have larger coverage than manual reviews. The scanners can analyze the code of the application, but they can also go out and scan third party libraries and dependencies that are out of the scope of a manual review. Despite these advantages the automated scanners lack context. SAST scanners are only able to scan the code itself. They cannot consider the context or logic behind the program. SAST scanners are also limited in scope. They can only identify vulnerabilities that are known to the tool. They will miss new or unknown vulnerabilities that are not in the tools database. Automated tests will be used throughout the development process to save time and allow for personnel to focus on more important tests.

The third leg of the security testing plan is automated dynamic testing. Just like SAST before there are automated tools to perform dynamic testing. These scanners are known as dynamic application security testing (DAST) scanners. Where SAST scanners have access to the code itself, DAST scanners take an “outside in” approach. They attack an application using techniques a malicious user would. After it completes these attacks, it looks for unexpected outputs and identifies security vulnerabilities. DAST scans happen after the application is finished in development. Meaning it doesn’t overlap with the manual and automatic source code reviews.

The fourth leg of the testing plan is penetration testing. Penetration testing, otherwise known as pen testing, is an exercise where a cyber-security expert attempts to find and exploit vulnerabilities in an application. The purpose of this exercise is to find weak spots in the systems defenses which a real attacker could exploit. There are many types of pen tests. However, for this security test plan we are going to use closed-box or black-box testing. These tests are performed with no extra knowledge of the application other than the name. The hacker must find the vulnerabilities on their own. These tests include brute-force attacks or SQL injections among other social engineering techniques. These tests provide invaluable understanding of weaknesses in the target application. This information is then passed on to the developers of the application where they can make improvements. Pen testing happens at the very end of development meaning it can't be relied on to catch all the possible security vulnerabilities.

In summary, the security testing plan is as follows. During the development of the application manual and automated source code reviews are to be performed. These reviews are preformed to catch problems before they become problems. Then when the application is mostly complete the automated dynamic testing can begin. These tests capture vulnerabilities that can't be captured by the previous static tests. Finally, before the application is released the pen testing can take place. By themselves each of the tests fail to cover everything. However, when used together they cover each other's weaknesses while providing their strengths.

References

About penetration testing. (2007, December 1). IEEE Journals & Magazine | IEEE Xplore. https://ieeexplore.ieee.org/abstract/document/4402456

Cong, N. T., Toan, L. H., & Minh, T. (2024). Tổng quan về phân tích tĩnh và phân tích động trong kiểm tra bảo mật ứng dụng. Journal of Military Science and Technology, 99, 1–11. https://doi.org/10.54939/1859-1043.j.mst.99.2024.1-11

Dencheva, L. (2022, December 2). Comparative analysis of Static application security testing (SAST) and Dynamic application security testing (DAST) by using open-source web application penetration testing tools. https://norma.ncirl.ie/5956/

Merkow, M. S., & Raghavan, L. (2011). Secure and resilient software: Requirements, Test Cases, and Testing Methods. CRC Press.

OWASP Application Security Verification Standard (ASVS) | OWASP Foundation. (n.d.). https://owasp.org/www-project-application-security-verification-standard/

Wen, S., & Katt, B. (2023). A quantitative security evaluation and analysis model for web applications based on OWASP application security verification standard. Computers & Security, 135, 103532. https://doi.org/10.1016/j.cose.2023.103532